Neurocomputing 610 (2024) 128586

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Robust filter pruning guided by deep frequency-features for edge intelligence

Yaguan Qian [a], Wenzhuo Huang [a], Qinqin Yu [a], Tengteng Yao [a], Xiang Ling [b], Bin Wang [c,*], Zhaoquan Gu [d], Yanchun Zhang [e]

[a] School of Science, Zhejiang University of Science and Technology, Hangzhou, 310023, China
[b] Institute of Software, Chinese Academy of Sciences, Beijing, 100190, China
[c] Zhejiang Key Laboratory of Artificial Intelligence of Things (AIoT) Network and Data Security, Hangzhou, 310052, China
[d] School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), Shenzhen, 518055, China
[e] School of Computer Science and Mathematics, Victoria University, Melbourne, VIC 8001, Australia

ARTICLE INFO

Communicated by X. Yang

Keywords:
Security vulnerability
Adversarial examples
Fourier transform
Robust pruning

ABSTRACT

With the popularity of mobile applications, smart terminals require the deployment of lightweight deep neural networks (DNNs). However, those lightweight DNNs deployed on resource-constrained edge devices, have serious security vulnerabilities to adversarial examples. Pruning methods have been widely used to obtain lightweight DNNs, which mainly focus on improving classification accuracy, but ignore robustness. To this end, we present a Fourier space-based robust pruning algorithm (FSRP). We find that a robust network pays more attention to the low-frequency region of the adversarial examples' feature maps, but ignores the high-frequency region. According to this finding, we design a filter robust indicator, a ratio of low-frequency components to high-frequency components of the feature map, to guide the pruning process. With this new robust pruning criterion, we adopt a strategy of local pruning that removes the filters with low robustness layer by layer in the model. Extensive experiments on CIFAR10/CIFAR100 and ImageNet show that the robust pruning accuracy is significantly improved under the FSRP robust pruning criterion. On CIFAR10, a pruning ratio of 90% on the VGG16 network still shows a 14.1% improvement in robust accuracy under auto attacks (AA).

1. Introduction

Artificial intelligence (AI) based on DNNs has achieved remarkable success in many application domains, such as image classification [1,2], object detection [3,4], and image analysis [5,6]. People are increasingly relying on the convenient services provided by AI, hoping to enjoy intelligent services anytime and anywhere. The maturity of deep learning techniques and edge computing systems [7,8], and the increasing demand for intelligent life promote the development and realization of *edge intelligence*. The so-called edge intelligence is to deploy DNNs to devices on the edge of networks (such as an intelligent camera of monitoring systems) to realize real-time applications.

However, the edge intelligence system faces two major new challenges at present [9]. First, the DNNs with millions of parameters greatly limit their deployment in resource-constrained edge devices [10], in other words, a lightweight network is required. Second, they are highly vulnerable to adversarial examples in open environments [11]. An *adversarial example* x' is a clean example x added by an elaborate small perturbation δ, *i.e.*, $x' = x + \delta$, where a DNN will give the wrong prediction on x' with high confidence, but humans are not fooled [12]. Therefore, how to create a robust and lightweight DNN for edge intelligence devices has attracted a lot of attention from the research community.

Several technologies are proposed to address the above challenges. Unlike traditional knowledge distillation [13], adversarial robust distillation [14] not only requires the student model to inherit accuracy from the robust teacher model but also inherit robustness. Huang et al. [15] proposed AdaAD (adaptive adversarial distillation), which searches for free matches for the distillation through the adaptive training process. Zhao et al. [16] proposed MTARD (multi-teacher adversarial robust distillation), which uses an adversarial teacher model and a clean teacher model to guide a lighter student model to obtain accuracy and robustness. In recent years, one of the approaches to model compression combining quantization and adversarial robustness has likewise achieved better results, *e.g.*, Ayaz et al. [17] designed deep quantization neural networks that are more accurate, robust, and suitable for resource-constrained devices by using the Jacobian regularization technique to improve the robustness of the model.

Although the above methods have achieved a perfectly robust and compact network, they lack flexibility in compression ratio. Robust

* Corresponding author.
E-mail address: wbin2006@gmail.com (B. Wang).

https://doi.org/10.1016/j.neucom.2024.128586
Received 22 December 2023; Received in revised form 24 July 2024; Accepted 11 September 2024
Available online 16 September 2024
0925-2312/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

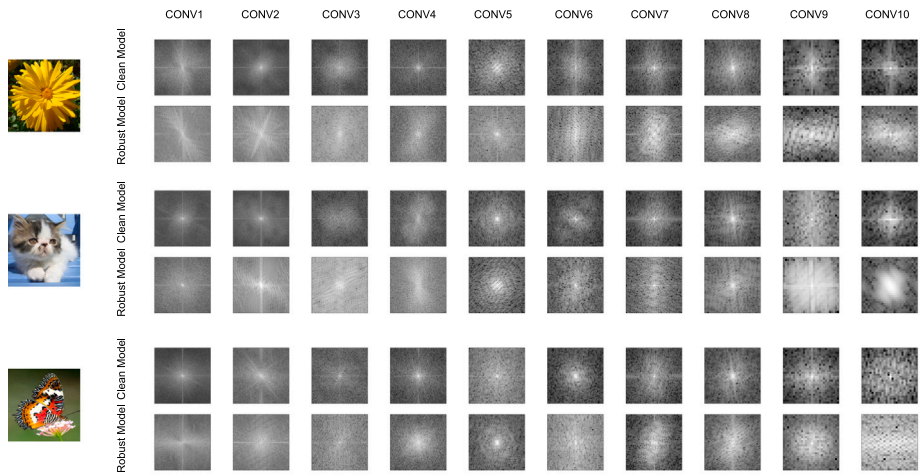

Fig. 1. Visualization of feature maps of layers 1–10 after 2D-DFT on the clean and robust models. Three adversarial examples are selected for visualization where the low-frequency components are in the middle and the high-frequency components are around. All adversarial examples are produced by PGD attack with 10 iterations on the same clean model.

pruning provides a flexible compression way for different edge intelligent applications. The so-called "robust pruning" involves incorporating considerations for the robustness of the pruned network in addition to network pruning. Its objective is not solely to eliminate redundant network connections but also to eliminate connections that compromise the network's robustness, resulting in a compact and resilient neural network. Network pruning can be divided into *weight pruning* (removing the unimportant weights) and *filter pruning* (removing the redundant filters/channels), also named weight pruning and filter pruning, respectively. However, the models pruned by weight pruning usually require specialized hardware devices or libraries for acceleration, which limits them from being deployed on commonly used devices of edge intelligence. On the contrary, filter pruning does not depend on specialized hardware devices or libraries for acceleration [18]. Hence, compared to weight pruning, filter pruning will be friendly to various edge devices. In this paper, we focus on filter pruning. Two important filter pruning methods, CHIP (channel independence-based pruning) [19] and GAL (generative adversarial learning) [20], are chosen for comparison. CHIP takes channel independence (an indicator to measure the correlation between different feature maps) as the importance of each channel and its corresponding filter and develops a low-cost and high-fine-grained filter pruning scheme. GAL generates pruning masks through generators, evaluates the performance of the model after pruning through discriminators, and combines soft masks to achieve the best structured pruning of convolutional networks.

We explore the most classic filter robust pruning methods. Sehwag et al. [21] demonstrated the early success of the Adv-LWM (lowest weight magnitude) robust pruning algorithm, which creates compact networks while maintaining accuracy and robustness. Although Adv-LWM pruning algorithm perform well in clean training, they exhibit significant performance degradation under adversarial training, with limited ability to defend against powerful adversarial attacks. Recently, Sehwag et al. [22] introduced the Hydra robust pruning algorithm, which leverages the objective function to steer the pruning training process. HYDRA utilizes weight importance scores to facilitate network pruning. However, it does not delve into the interpretability of weight importance for enhancing robustness. We propose a new robust filter pruning algorithm named FSRP (Fourier space-based robust pruning) from the view of frequency aimed to suppress the adversarial perturbation in deep feature space. In this method, the filters' robustness criterion is a core component. We aim to examine the robustness of the filters through the frequency domain. Yin et al. [23] showed that traditional DNNs are sensitive to high-frequency changes in images, while most DNNs are robust to low-frequency changes. Nonetheless, we cannot definitively ascertain if this conclusion remains valid within the deep feature space of the DNN. Since we conduct pruning within the internal hidden layers of the DNN, we should further explore the relationship between robustness and the feature maps generated by the filters in the frequency domain. Thus, we can finally build the filters' robustness criterion.

We investigate the difference between robust models[1] and clean models[2] in the deep feature space. Fig. 1 shows the feature maps generated by three adversarial examples produced by PGD attack with 10 iterations on layers 1–10 of the clean model and the robust model (both VGG16) after the two-dimensional discrete Fourier transform (2D-DFT), where the low-frequency components are in the middle and the high-frequency components are around. In contrast to the clean model, the adversarial examples are significantly expanded in the low-frequency component of the robust model. We infer that the robust model pays more attention to the low-frequency components in the deeper features of the adversarial examples. Therefore, we define the ratio of the low-frequency component to the high-frequency component of a feature map as a measure of the robustness of the corresponding filter. With this ratio, we construct a new robust pruning criterion to guide the pruning algorithm, *i.e.*, FSRP. Furthermore, we adopt a strategy of local pruning that removes the non-robust filters. Meanwhile, we employ adversarial training to further improve the network's robustness.

The main contributions are summarized as follows:

- We gain insight into the phenomenon that occurred in the deep feature space under adversarial attacks, and reveal that the robust model with adversarial training focuses more on the low-frequency regions of the feature map. This finding helps us to find a proper robust pruning criterion.
- We propose a new measure to identify non-robust feature maps via the view of the frequency, which helps us to better identify non-robust filters. By pruning these non-robust filters, the network becomes more robust and compacted as well.
- Extensive experiments show that FSRP outperforms baseline and general filter pruning methods. FSRP achieves a 90% parameter reduction and still improves robust accuracy by 14.1% under AA.

The rest of the paper is structured as follows: In Section 2, we review adversarial attacks and defenses, meanwhile, introducing lightweight neural networks and robustness. In Section 3, we formally describe the concepts of DNNs and adversarial examples. We also provide the

[1] A robust model is one trained with adversarial examples to achieve robustness, which is currently the most effective way to obtain robustness.

[2] A clean model is one trained standardly with clean examples.

background of 2D-DFT, which is used in our method. Section 4 demonstrates our pruning method, FSRP, in detail. Section 5 provides the experimental results. We discussed the limitations in Section 6. Finally, a conclusion is given to end this paper in Section 7.

2. Related work

2.1. Adversarial attacks and defenses

Szegedy et al. [12] first observed the phenomena of adversarial examples. They craft adversarial examples by finding the minimum loss function additive perturbation term that causes the neural network to misclassify. The fast gradient sign method (FGSM), which was pioneered by [11], uses a step size to regulate L_∞ the perturbation's norm. The basic iterative method of projected gradient descent (PGD) proposed by [24] can be viewed as an iterative version of FGSM. More recently, Croce and Hein [25] introduced AA, which combines two extensions of PGD with FAB [26] and Square Attack [27] to automatically evaluate defense methods without tuning parameters. These approaches are also called white-box attacks since the adversary has full knowledge of DNNs. Except for white-box attacks, there are also many ways to explore the transferability of adversarial examples for black-box attacks. Liu et al. [28] was the first to study the transferability of targeted adversarial examples. Dong et al. [29] showed that iterative attack methods incorporating the momentum term achieved better transferability. Further, Xie et al. [30] boosted the transferability of adversarial examples by creating diverse input patterns with random resize and random padding.

At the same time, many defense methods were proposed, and the current defense methods roughly include randomization, detection, and adversarial training. Liu et al. ([31] adds randomization to the weights of the model using a Bayesian component. The well-designed detectors can detect adversarial examples. Gong and Wang [32] built simple binary classifiers that separate adversarial examples from clean examples. The most effective defense at present is adversarial training (AT) [33]. Goodfellow et al. [11] proposed to generate adversarial examples with FGSM to perform adversarial training. Next, the max–min formula introduced by [24] as the standard adversarial expression for training. To achieve further improvements in AT, Zhang et al. [34] proposed a trade-off between robustness and accuracy (TRADES). ang et al. [35] proposed the misclassification aware adversarial training (MART). By comparing the three most commonly used adversarial training methods, TRADES training method can achieve comprehensive optimization.

2.2. Lightweight neural networks and robustness

DNNs are over-parametric and susceptible to adversarial attacks, so it is a new trend to consider model robustness and computational overhead in the model design process. Huang et al. [36] found that reducing the capacity at the end of the network can improve the adversarial robustness. There are works [37–39] that reveal that proper network pruning can enhance the robustness of the network, so finding the law of adversarial robustness in the network structure is the key to solving the problem.

Traditional robust pruning work, such as a parallel adversarial training and weighted pruning framework proposed by [37] seeks to compress robust models. Gui et al. [38] further incorporated pruning and several other techniques into a unified optimization framework to achieve high compression ratios while maintaining high robustness. Guo et al. [40] theoretically demonstrated that appropriately increasing the weight sparsity means that cleanly trained models are more robust. However, sparsification methods not only cause instability in the network [41], but excessive sparsity can also reduce the robustness of the network [40]. In fact, weight pruning requires specific hardware libraries for acceleration, which is difficult to realize in existing applications.

In order to enable a wide range of applications of robust pruning, this paper focuses on exploring structured pruning algorithms. The robust pruning algorithm Adv-LWM [21] and the improved HYDRA algorithm [22] proposed by Sehwag et al. with classical filter pruning techniques that consider robustness. GAL [20] proposes an adversarial learning framework using category-free label generation and solves the objective function optimization problem using fast iterative threshold shrinkage algorithms that steadily remove redundant structures. CHIP [19] uses channel in-dependence to represent the importance of each feature mapping (and its corresponding filter). However, the above-mentioned structured pruning only innovates in the training method or objective function, and does not study the potential robustness of the network.

Networks with lightweight structures have been proposed to address the challenges of resource-constrained environments. These architectures incorporate various design principles to reduce model complexity while maintaining satisfactory performance. SqueezeNet [42] adopts 1×1 convolution kernels for efficient feature extraction and draws inspiration from the inception structure [43]. ShuffleNetV2 [44] reduces parameters by group convolution. MobileNetV1 [45] uses depth-separable convolution. MobileNetV2 [46] has an inverted residual structure. EfficientNet [47] scales the model's width. These are examples of networks designed with lightweight architectures.

3. Preliminary

3.1. DNNs and adversarial examples

A deep neural network can be generally expressed as a mapping function $f : \mathcal{X} \to \mathcal{Y}$, where $\mathcal{X}$ is an input space and $\mathcal{Y}$ is a label space. Assume a pre-trained DNN model f has a set of L convolutional layers, and C^i is the ith convolutional layer. The parameters in layer C^i can be represented as a set of 3-D filters $W_{C^i} = \left\{ w^i_1, w^i_2, \dots, w^i_{N_i} \right\} \in \mathbb{R}^{N_i \times N_{i-1} \times K_i \times K_i}$, and the jth filter in layer C^i is denoted by $w^i_j \in \mathbb{R}^{N_{i-1} \times K_i \times K_i}$, where $i = 1, 2, \dots, L$, N_i represents the number of filters in layer C^i, $N_0 = 3$ and $K_i \times K_i$ denotes the kernel size. The outputs of the filters, *i.e.*, feature maps, in layer C^i are denoted by $O^i = \left\{ o^i_1, o^i_2, \dots, o^i_{N_i} \right\} \in \mathbb{R}^{N_i \times h_i \times w_i}$, where the jth feature map $o^i_j \in \mathbb{R}^{h_i \times w_i}$ is generated by w^i_j. Here, h_i and w_i are the height and width of the feature map, respectively.

Next, we formally define adversarial examples. Let x be a clean image, and y is its ground-truth label. Let δ be a well-designed perturbation and $x' = x + \delta$ such that $\|\delta\|_p < \epsilon$. We say x' is an adversarial example if $f(x') \neq y$. Here $\|\cdot\|_p$ is the p-norm ($p = 1, 2, \infty$). The purpose of imposing this constraint on the perturbation δ is to guarantee it is imperceptible to humans. Accordingly, crafting adversarial examples to fool the DNN is called adversarial attacks.

3.2. 2D discrete Fourier transform

In this paper, we mainly use the Fourier transform as an analysis tool. 2D-DFT is a transformation method that converts an image from the spatial domain to the frequency domain, where the image is a two-dimensional matrix. Converting an image from the spatial domain to the frequency domain enables more intuitive observation and processing of the image, and is also more conducive to the operation of frequency domain filtering.

Let $F(\cdot,\cdot)$ be a 2D-DFT in the following form:

$$F(u,v) = \sum_{x=0}^{h_i-1} \sum_{y=0}^{w_i-1} f(x,y) e^{-\mathrm{j}2\pi\left(\frac{xu}{h_i} + \frac{yv}{w_i}\right)}, \tag{1}$$

where $u = 0, 1, \dots, h_i - 1$, $v = 0, 1, \dots, w_i - 1$, $\mathbf{j}$ represents an imaginary unit, h_i is the number of rows of o^i_j, and w_i is the number of columns

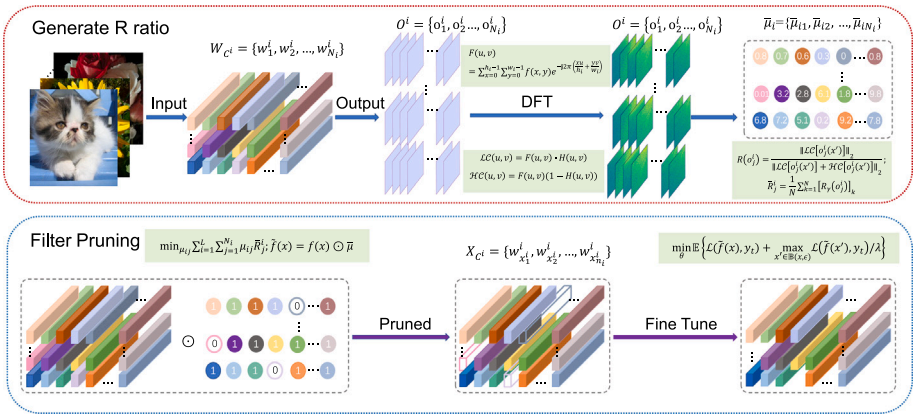

Fig. 2. Illustration of our robust filter pruning process (FSRP). The adversarial examples are fed into the pre-trained CNN. The resulting output feature map is performed by DFT and the low-frequency and high-frequency components are separated. The ratio of the low-frequency component to the high-frequency component is used to guide the CNN for filter pruning. Finally, the fine-tuned CNN becomes more robust and compact.

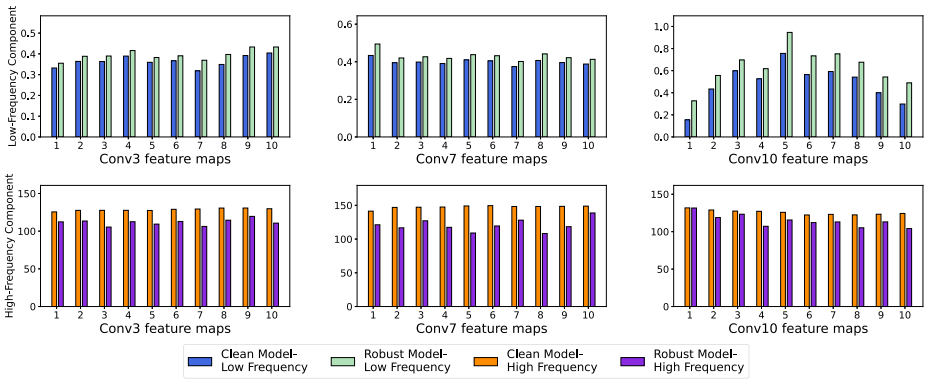

Fig. 3. The low-frequency average component and the high-frequency average component of the feature map are compared between adversarial examples processed by the clean model and the robust model. Following the input of adversarial examples created through the PGD attack with 10 iterations into both the clean and robust models, we randomly identify the 3rd, 7th, and 10th convolutional layers to analyze the low-frequency and high-frequency average components of the resulting feature maps produced by a common filter. The top row displays the average low-frequency components, while the bottom row illustrates the average high-frequency components across these feature maps.

of o_j^i. Each layer's output o_j^i is used as input to the 2D-DFT. f is the inverse transformation of F, and its form is as follows:

$$f(x,y) = \frac{1}{h_i w_i} \sum_{u=0}^{h_i-1} \sum_{v=0}^{w_i-1} F(u,v) e^{j2\pi\left(\frac{xu}{h_i}+\frac{yv}{w_i}\right)}, \tag{2}$$

where $x = 0, 1, \ldots, h_i - 1,\ y = 0, 1, \ldots, w_i - 1$.

The low-frequency component and high-frequency component are calculated by Eqs. (3) and (4), respectively:

$$\mathcal{LC}(u,v) = F(u,v)H(u,v), \tag{3}$$

$$\mathcal{HC}(u,v) = F(u,v)(1 - H(u,v)), \tag{4}$$

where

$$H(u,v) = \begin{cases} 1, & \text{if } \sqrt{(u-u_0)^2 + (v-v_0)^2} \leqslant \zeta \\ 0, & \text{otherwise} \end{cases}, \tag{5}$$

(u_0, v_0) are the coordinates of the centroid of the 2D-DFT transformed image, and $\zeta \in [0,1]$ is the cutoff frequency, which is used to retain the low-frequency component. We take the median value of ζ as 0.5 in this paper. $H(u,v)$ is a filter function that is used to select the low-frequency component. $(1 - H(u,v))$ denotes the complement of the ideal low-pass filter, *i.e.*, the selection of the high-frequency component in the frequency domain.

4. Methodology

Ilyas et al. [48] claimed that adversarial vulnerability is caused by non-robust features. The reason for the existence of adversarial examples is due to the non-robust features of clean examples. These non-robust features can improve the prediction performance, but can be exploited by an attacker, *i.e.*, adding perturbations to non-robust features can effectively mislead CNNs. On the contrary, if we remove the perturbations of the non-robust features, we can defend against adversarial attacks. Li et al. [49] further explored the concept of non-robust features in various paradigms. It is recognized that adversarial vulnerability does not solely originate from input data or features, and true robustness can only be achieved by addressing vulnerabilities in both the data and the models. In this paper, we observe the model's vulnerability through non-robust features. Yang et al. [50] extracted the non-robust features in the image data through the embedding vector after the model convolution layer, improving the model's robustness while ensuring clean accuracy. However, the above work only explores the non-robust features in the *input space* (the original image space), and isolating these non-robust features from the image is non-trivial in fact. Different from the previous works that focus on removing perturbations in the input space, we hope to obtain a robust network architecture by pruning the non-robust filters in the *deep feature space*. The procedure of our method is illustrated in Fig. 2.

4.1. Non-robust feature maps

The perturbation signals added to clean examples will be amplified by some filters across all convolutional layers. These amplified perturbations are involved in the corresponding feature maps. However, directly discovering these non-robust feature maps is difficult. The

Fourier transform provides another way to gain insight into these feature maps. We use the low-frequency and high-frequency components defined in the literature [51] to conduct the Fourier analysis on feature maps to observe non-robust ones. Although [23] studied the relationship between the model robustness and the image frequencies in the input space, they did not investigate it in the deep feature space. Therefore, their conclusions cannot be directly used as a basis for internal pruning of DNNs. We need to further explore the relationship between deep feature maps and model robustness.

Fig. 3 shows the low-frequency and high-frequency components of the feature maps generated in the clean model and robust model, respectively, when inputting adversarial examples. Note that the frequency domain components here are the average frequency domain components of the feature map at the same locations. From Fig. 3, we observe that in the robust model, the low-frequency region achieves higher gains, whereas in the clean model, the opposite holds for the feature map produced by the same filter. In other words, the robust model pays more attention to the low-frequency region and less notice to the high-frequency region of the feature map in the face of adversarial examples. This explains why the model with adversarial training becomes robust, while the clean model is vulnerable to adversarial examples. Therefore, if we hope to improve the robustness of a clean model, we should suppress these feature maps with higher gains achieved in low-frequency regions.

To this end, we distinguish robust and non-robust feature maps. The energy in the feature maps' high-frequency and low-frequency regions can be expressed using the L_2 norm, while the ratio between them can reflect whether the energy of the corresponding filter is concentrated in high-frequency or low-frequency regions, thereby enabling us to determine which filters to prune. Since we perform pruning in this paper, non-robust feature maps are interesting for us. We formally define nor-robust feature maps as follows:

Definition 1 (*γ-non-robust Feature Maps*)**.** Let $x \in \mathcal{D}$ be a clean example and its adversarial example is denoted by $x' \in \mathcal{D}'$. $o_j^i(x')$ denotes the feature map generated by the jth filter in layer C^i when input x'. We obtain a ratio between low and high frequencies of the feature map:

$$R\left(o_j^i\right) = \left\|\mathcal{LC}\left[o_j^i(x')\right]\right\|_2 / \left(\left\|\mathcal{LC}\left[o_j^i(x')\right]\right\|_2 + \left\|\mathcal{HC}\left[o_j^i(x')\right]\right\|_2\right). \tag{6}$$

If $R\left(o_j^i\right) < \gamma$, we claim the feature map o_j^i is γ-non-robust and simply denote its ratio by R_γ.

The above definition implies that in non-robust feature maps, energy is more concentrated in the high-frequency region compared to the low-frequency region. Since the clean models' attention is more focused on the high-frequency region, adversarial noise on the high-energy high-frequency region is more likely to impact the model's robustness.

4.2. Non-robust filters

In this paper, we hope to prune the filters to improve the robustness of a clean model. As we know, a feature map is an output of a filter representing a kind of specific feature. Therefore, the feature map's robustness reflects the filter's robustness. In other words, a non-robust filter will amplify adversarial perturbations and output a non-robust feature map.

Definition 2 (*Non-robust Filters*)**.** Suppose there are N adversarial examples that have R_γ value on a filter, we define the average of R_γ as a metric to measure the non-robustness of the filter:

$$\bar{R}_j^i = \frac{1}{N}\sum_{k=1}^{N}\left[R_\gamma\left(o_j^i\right)\right]_k, \tag{7}$$

where o_j^i is the feature map generated by the jth filter in the ith layer and $\left[R_\gamma(\cdot)\right]_k$ represents the R_γ value of the kth adversarial example.

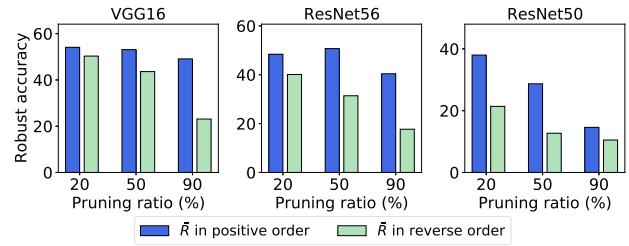

Fig. 4. Robust accuracy comparison of the network pruned according to $\bar{R}$ in positive order and reverse order.

We empirically verify the rationality of using the $\bar{R}$ value in Eq. (7) as the robustness measure of a filter. The low-frequency and high-frequency components of the feature maps were obtained from VGG16, ResNet56, and ResNet50 models with adversarial training. And then, the $\bar{R}$ values of the corresponding filters were obtained from Eqs. (6) and (7). Finally, The $\bar{R}$ values are sorted in positive order (from smallest to largest) and their corresponding filters are then removed with pruning ratios of 20%, 50%, and 90%, which means that the non-robust filters are pruned first. Then all filters' $\bar{R}$ values are sorted in reverse order (from large to small), and the filters are pruned with the previous pruning ratios. The pruned models are evaluated with the PGD attack with 100 iterations. The results shown in Fig. 4 indicate that the higher the low-frequency proportion of the feature maps as mentioned above, the corresponding filters are more robust because we remove the weak robust filters to make the DNN pay more attention to the strong robust filters. On the contrary, if we reserve more filters with low $\bar{R}$ value, more weak robust filters will focus on the high-frequency regions with strong vulnerability in the feature map, which will lead to the reduction of robust accuracy.

Since Eq. (7) shows that our robustness measure for a filter is based on statistical significance, we will discuss the effect of test data size N on $\bar{R}$ below. We use different batches of CIFAR10 to evaluate the statistical robustness of $\bar{R}$. As shown in Fig. 5, it can be observed that the $\bar{R}$ value remains almost constant regardless of the number of batches.

4.3. Robust pruning

Though global pruning of neural networks can produce better results, this can also lead to layer collapse [52]. We used a local pruning strategy to remove the filters with low robustness layer by layer in the model, which can be formulated as an optimization problem:

$$\begin{aligned} &\min_{\mu_{ij}} \sum_{i=1}^{L}\sum_{j=1}^{N_i} \mu_{ij}\,\bar{R}_j^i, \\ &\text{s.t.} \sum_{i=1}^{L}\sum_{j=1}^{N_i} \mu_{ij} = n_i, \end{aligned} \tag{8}$$

where we use the $\bar{R}_j^i$ defined in Eq. (7) as the filter robustness measure. If the filter is robust, the value of μ_{ij} is 0. If the filter is non-robust, the value of μ_{ij} is 1. The constraint indicates the number of filters being pruned $n_i = p \times N_i$. Minimizing the objective function in Eq. (8) is equivalent to removing the first n_i non-robust filters in C^i, so this paper solves the pruning problem by sorting the $\bar{R}$ values from smallest to largest. Our pruning process is demonstrated as follows.

Suppose a CNN has a total of L convolutional layers, and the filters W_{C^i} in ith convolutional layer C^i outputs feature maps denoted as O^i, with N_i filters per layer and pruning ratio of p. (1) According to Eq. (6) and (7), calculate the $\bar{R}$ value of all feature maps O^i in the convolution layer C^i, and obtain the set $\bar{\mathcal{R}}^i = \left\{\bar{R}_1^i, \bar{R}_2^i, \ldots, \bar{R}_{N_i}^i\right\} \in \mathbb{R}^{N_i}$; (2) Sort the $\bar{R}$ value of $\bar{R}^i$ in positive order, formed a set $\hat{\mathcal{R}}^i = \left\{\hat{R}_1^i, \hat{R}_2^i, \ldots, \hat{R}_{N_i}^i\right\} \in \mathbb{R}^{N_i}$; (3) Set the pruning ratio p, determining the number of non-robust

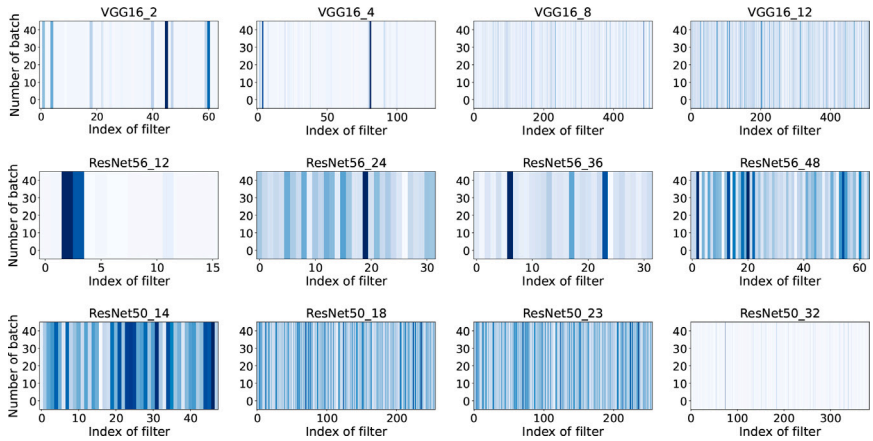

Fig. 5. Average R-values of the adversarial examples generated using different batches of CIFAR10 via PGD attack with 10 iterations on different robust models of feature maps. For each subplot, the horizontal axis represents the index of the filter and the vertical axis represents the number of batches. The size of each batch is 64. different colors represent different $\bar{R}$ values. As shown in the figure, the $\bar{R}$ values of the same filter (each column of the subgraph) hardly change (same color) for different numbers of batches. Therefore, the average R ratio of each feature map in different architectures can be effectively estimated even for a small number of images.

filters $n_i = p \times N_i$ and the number of robust filters $m_i = (1-p) \times N_i$; (4) According to Eq. (8), divide non-robust and robust sets. The set of non-robust filters is $X_{C^i} = \left\{ w^i_{x^i_1}, w^i_{x^i_2}, \ldots, w^i_{x^i_{n_i}} \right\}$, and the set of robust filters is $Y_{C^i} = \left\{ w^i_{y^i_1}, w^i_{y^i_2}, \ldots, w^i_{y^i_{m_i}} \right\}$, where $X_{C^i} \cup Y_{C^i} = W_{C^i}$, $X_{C^i} \cap Y_{C^i} = \varnothing$, and $n_i + m_i = N_i$; (5) Create mask vector $\bar{\mu}_i = \left\{ \bar{\mu}_{i1}, \bar{\mu}_{i2}, \ldots, \bar{\mu}_{iN_i} \right\}$. The non-robust filter mask $\bar{\mu}_{ij}$ is set to 0, and the robust filter mask $\bar{\mu}_{ij}$ is set to 1. The pruning process is executed cyclically until all convolutional layers have performed the pruning operation.

When the mask vectors of all convolutional layers are obtained, the final pruned structure can be expressed as the Hadamard product of the original network and the mask. We define the pruning process as:

$$\tilde{f}(x) = f(x) \odot \bar{\mu}, \tag{9}$$

where $\bar{\mu}$ is the mask matrix, and column i represents the $\bar{\mu}_i$ mask vector, $f(x)$ represents the original model, and $\tilde{f}(x)$ represents the pruned model. The specific process is presented in Algorithm 1.

4.4. Fine tuning

To ensure both robust and clean accuracy, we retrain the pruned model $\tilde{f}(x)$. The loss function for retraining is composed of two terms: (1) The first term is to minimize the empirical risk of clean examples, that is, to guarantee the clean accuracy; (2) The second regularization term uses adversarial examples to encourage output smoothing and pushes the decision boundary far away from boundary examples to further improve adversarial robustness. To balance clean accuracy with robust accuracy, we suggest minimizing the following:

$$\min_{\theta} \mathbb{E} \left\{ \mathcal{L}\left(\tilde{f}(x), y_t\right) + \max_{x' \in \mathbb{B}(x,\epsilon)} \mathcal{L}\left(\tilde{f}\left(x'\right), y_t\right) / \lambda \right\}, \tag{10}$$

where x is the clean example, y_t is the corresponding ground-truth label, x' is the adversarial example, $\mathbb{B}(x,\epsilon)$ is a norm-ball representing the adversarial example space, $\mathcal{L}(\cdot)$ is the cross-entropy loss function, and λ is the tuning parameter.

5. Experiments

In this section, we evaluate the robustness of our pruned model against powerful attacks. The pruned model with our FSRP achieves more robust defense performance while maintaining clean accuracy.

Algorithm 1 Algorithm Description of FSRP.

Require: Robust CNN f, Dataset batchsize N, pruning ratio p
Ensure: Robust pruned network $\tilde{f}$
  // *Phase I:* Generate $\bar{R}$ ratio
  for $i = 1$ to L **do**
    for $j = 1$ to N_i **do**
      According to Eqs. (3) and (4), calculate the low-frequency component $\mathcal{LC}\left[o^i_j(x')\right]$ and the high-frequency component $\mathcal{HC}\left[o^i_j(x')\right]$ of the feature maps $o^i_j(x')$;
      According to Eqs. (6) and (7), calculate the $\bar{R}^i_j$ value of the feature maps $o^i_j(x')$;
    end for
  end for
  // *Phase II:* Filter Pruning
  for $i = 1$ to L **do**
    for $j = 1$ to N_i **do**
      Obtain the sorted $\hat{R}^i_j$ by $\bar{R}^i_j$;
      According to Eq. (8) and p, obtain the set of non-robust filters $X_{C^i} = \left\{ w^i_{x^i_1}, w^i_{x^i_2}, \ldots, w^i_{x^i_{n_i}} \right\}$, the set of robust filters $Y_{C^i} = \left\{ w^i_{y^i_1}, w^i_{y^i_2}, \ldots, w^i_{y^i_{m_i}} \right\}$, and the mask vector $\bar{\mu}_i = \left\{ \bar{\mu}_{i1}, \bar{\mu}_{i2}, \ldots, \bar{\mu}_{iN_i} \right\}$;
    end for
  end for
  $\tilde{f}(x) = f(x) \odot \bar{\mu}$;
  // *Phase III:* Trades off Robustness and Accuracy
  $\min_{\theta} \mathbb{E} \left\{ \mathcal{L}\left(\tilde{f}(x), y_t\right) + \max_{x' \in \mathbb{B}(x,\epsilon)} \mathcal{L}\left(\tilde{f}\left(x'\right), y_t\right) / \lambda \right\}$.

5.1. Experimental settings

The dataset and model specifications, as well as training and evaluation details, are described as follows.

Datasets and models: In this section, we evaluate the FSRP framework, based on the CIFAR10/100 [53], ImageNet30 [54], and ImageNet datasets. On CIFAR and ImageNet30, we conduct the FSRP algorithm on two classic types of network structures: VGG16 [55] with a plain structure and ResNet56 [1] with residual blocks. On ImageNet, we use ResNet50 to perform FSRP.

Table 1
Comparing FSRP with Adv-LWM and HYDRA robust pruning results on VGG16 and ResNet56 learned on CIFAR10.

Model	Adv. training	Attack type	Pre-trained	20%			50%			90%		
				Adv-LWM	HYDRA	FSRP	Adv-LWM	HYDRA	FSRP	Adv-LWM	HYDRA	FSRP
VGG16	SAT	Clean	82.9	80.4	82.9	**83.8**	76.1	78.9	**80.1**	74.9	76.4	**78.8**
		FGSM	53.6	55.9	57.4	**59.4**	49.3	51.9	**58.0**	41.9	46.5	**54.6**
		PGD^{20}	41.3	42.3	44.2	**54.1**	41.2	44.3	**53.1**	38.7	42.6	**49.1**
		PGD^{100}	40.6	42.3	44.1	**53.9**	41.2	44.1	**53.0**	38.7	42.4	**49.0**
		AA	38.5	39.7	40.3	**51.9**	39.1	40.3	**50.0**	35.8	36.2	**46.9**
	TRADES	Clean	83.0	81.0	79.9	**84.0**	78.0	79.6	**83.0**	72.0	74.6	**81.1**
		FGSM	54.1	57.9	56.3	**65.5**	50.4	50.7	**60.6**	43.7	46.8	**56.7**
		PGD^{20}	54.0	48.0	49.6	**58.9**	43.8	44.1	**57.1**	38.6	40.5	**49.6**
		PGD^{100}	54.0	48.1	49.6	**58.9**	43.6	44.0	**57.0**	38.6	40.5	**49.7**
		AA	44.9	42.3	43.0	**58.9**	40.1	40.9	**52.7**	35.1	36.9	**49.8**
	MART	Clean	80.0	79.4	81.2	**83.6**	76.2	79.1	**81.7**	69.0	74.8	**79.8**
		FGSM	57.0	58.2	58.9	**63.7**	51.2	52.8	**62.1**	41.6	47.2	**55.5**
		PGD^{20}	54.0	48.0	49.6	**58.9**	43.8	44.1	**57.1**	38.6	40.5	**49.6**
		PGD^{100}	54.0	48.1	49.6	**58.9**	43.6	44.0	**57.0**	38.6	40.6	**49.7**
		AA	45.1	41.6	42.4	**57.2**	38.6	40.0	**54.9**	32.1	34.0	**48.1**
ResNet56	SAT	Clean	83.9	84.1	84.6	**85.3**	78.6	79.4	**82.3**	70.1	69.8	**75.6**
		FGSM	55.8	54.5	**56.2**	56.0	47.1	52.6	**60.0**	37.4	41.9	**45.8**
		PGD^{20}	44.3	43.7	45.2	**48.4**	39.0	41.7	**50.7**	32.8	33.7	**40.4**
		PGD^{100}	43.8	43.8	45.2	**48.3**	38.9	41.6	**50.7**	32.7	33.8	**40.5**
		AA	41.8	40.6	41.7	**46.2**	32.5	34.7	**48.9**	29.6	30.9	**37.7**
	TRADES	Clean	82.0	83.1	84.6	**84.9**	79.4	79.9	**83.2**	70.1	71.3	**78.2**
		FGSM	59.0	58.6	59.8	**61.9**	49.2	53.8	**60.8**	41.4	43.9	**47.8**
		PGD^{20}	50.3	50.4	52.0	**54.3**	40.6	43.7	**56.7**	34.5	35.6	**41.2**
		PGD^{100}	49.7	50.3	51.9	**54.2**	40.5	43.5	**56.5**	34.5	35.5	**41.1**
		AA	45.0	44.3	46.2	**51.4**	35.4	37.9	**50.4**	28.1	30.0	**38.5**
	MART	Clean	80.0	81.4	82.3	**85.1**	78.4	79.2	**82.3**	69.8	68.4	**75.4**
		FGSM	58.0	**57.9**	56.6	52.0	50.3	54.0	**54.9**	45.3	46.2	**54.0**
		PGD^{20}	55.0	54.3	**56.3**	47.0	43.2	41.9	**48.3**	38.9	40.3	**48.4**
		PGD^{100}	53.0	54.1	**56.3**	47.0	43.1	41.8	**48.2**	38.6	40.1	**48.2**
		AA	45.1	43.2	41.7	**46.0**	38.7	36.9	**46.1**	31.2	34.6	**45.1**

Our experiment selected four different types of architecture widely used in edge intelligence as baselines: (1) Filter pruning method considering robustness, including ADV-LWM [21] and HYDRA [22]. The results are shown in Tables 1, 2, 3, and 4. (2) Filter pruning methods without considering robustness, including GAL [20] and CHIP [19]. The results are shown in Table 5. (3) Search network, including PNAS [56], NasNet [57], AmoebaNet [58], DARTS [59], P-DARTS [60], PC-DARTS [61], RACL [62], RobNet-free [63], DSRNA [64], and AdvRush [65]. The results are shown in Table 6. (4) Lightweight network, including SqueezeNet [42], ShuffleNetV2 [44], MobileNetV1 [45], MobileNetV2 [46], and EfficientNet [47]. The results are shown in Table 6.

Training settings: Since our goal is to prune on robust models, the three most classic adversarial training methods, AT [24], TRADES [34], and MART [35] are used as pre-training methods for our model. For each, we use the PGD attack with 10 iterations and a perturbation strength $\epsilon = 8/255$ with step size $\alpha = 2/255$. We adopt a weight decay of 0.0005 and a learning rate of 0.1. SGD is adopted as the gradient optimization algorithm with an initial learning rate of 0.9 and a momentum of 0.1. For TRADES and MART, we follow the originally proposed regulation of $\lambda = 6.0$ and $\lambda = 5.0$, respectively.

We use local pruning as the pruning strategy and use three representative pruning ratios of 20%, 50%, and 90%. We randomly select 1000 images to estimate each filter $\bar{R}$. On CIFAR, we train for 200 epochs and fine-tune 20 epochs. On ImageNet30, we train 40 epochs and fine tune 10 epochs. On ImageNet, we train 25 epochs and fine tune 5 epochs.

Performance metric: We provide top-1 accuracy of pruned models on CIFAR, ImageNet30 and ImageNet. To evaluate robust accuracy, we adopt (1) FGSM attack with 20 iterations, $\epsilon = 8/255$. (2) PGD^{20} and PGD^{100} refer to PGD attack with 20 and 100 iterations, $\epsilon = 8/255$. (3) AA attack with 20 iterations, $\epsilon = 8/255$.

5.2. Results on CIFAR

We evaluate our method on VGG16 and ResNet56 for CIFAR. The results are shown in Tables 1 and 2.

Effect of different adversarial training on model robustness: Regardless of which pre-training method is used, the models achieve a substantial improvement in robust accuracy with FSRP while maintaining a reasonable range of clean accuracy. On CIFAR10, most of the clean and robust accuracies of the model under TRADES training are better than SAT and TRADES. On the CIFAR100, the AT has slightly better robust accuracy and the TRADES has better clean accuracy. Overall, the model under TRADES pre-training is more stable in terms of clean accuracy and robust accuracy. We, therefore, pre-trained the model on the ImageNet30 and ImageNet datasets referring to Tables 3 and 4, again using the same settings as TRADES.

Comparison with filter pruning considering robustness: Adv-LWM and HYDRA are classical robust pruning methods. According to [21,22], their methods are also applicable to filter pruning and show significant performance. They can maintain both clean accuracy and robust accuracy at a pruning ratio of 20%. However, at a pruning ratio of 50%, there is a slight decrease in clean accuracy and robust accuracy. At a pruning ratio of 90%, their performance significantly deteriorates. Adv-LWM and HYDRA rely solely on the objective function to guide the pruning process without further exploring the underlying principles of robustness in deep network features. In most cases, FSRP significantly outperforms Adv-LWM and HYDRA. FSRP improves model robustness without sacrificing clean accuracy. After the non-robust filters are pruned by FSRP, the robust model will focus on the robust filters, as a matter of course, its robust accuracy is improved.

Comparison with filter pruning without considering robustness: We compare with two filter pruning methods that do not consider robustness, including GAL and CHIP, and the experimental results are shown in Table 5. FSRP demonstrates the best performance in terms of both robust accuracy and clean accuracy. At three pruning ratios

Table 2
Comparing FSRP with Adv-LWM and HYDRA robust pruning results on VGG16 and ResNet56 learned on CIFAR100.

Model	Adv. training	Attack type	Pre-trained	20%			50%			90%		
				Adv-LWM	HYDRA	FSRP	Adv-LWM	HYDRA	FSRP	Adv-LWM	HYDRA	FSRP
VGG16	SAT	Clean	54.1	50.0	52.9	**55.1**	45.3	46.9	**49.2**	36.5	40.8	**45.8**
		FGSM	31.3	32.8	34.6	**40.8**	29.1	32.9	**37.7**	21.9	24.6	**26.7**
		PGD^{20}	28.4	30.7	29.7	**39.8**	27.6	31.4	**39.9**	18.4	**19.0**	17.5
		PGD^{100}	29.0	30.5	29.7	**39.7**	27.5	31.4	**39.9**	18.2	**18.6**	17.5
		AA	25.9	26.1	26.0	**33.6**	24.3	22.6	**31.7**	16.2	**15.7**	14.3
	TRADES	Clean	54.1	55.3	**56.0**	53.8	50.6	51.8	**54.5**	40.8	40.4	**49.9**
		FGSM	29.9	30.5	31.2	**35.1**	30.1	31.6	**31.7**	27.6	27.4	**33.4**
		PGD^{20}	28.3	27.2	29.4	**32.9**	24.9	26.6	**27.9**	20.2	20.8	**30.2**
		PGD^{100}	28.3	27.1	29.3	**31.8**	24.2	26.5	**27.9**	20.1	20.3	**29.1**
		AA	22.4	23.8	25.1	**29.4**	20.2	23.4	**25.6**	15.6	17.1	**24.1**
	MART	Clean	43.2	41.6	42.9	**46.8**	34.2	39.9	**42.8**	29.6	31.5	**40.9**
		FGSM	31.0	25.4	**29.2**	28.3	20.2	21.7	**30.3**	16.7	17.1	**33.2**
		PGD^{20}	27.0	21.6	25.6	**27.1**	18.2	19.3	**27.9**	15.9	16.1	**21.3**
		PGD^{100}	27.0	21.5	25.8	**27.1**	18.0	19.2	**27.8**	15.4	15.8	**21.4**
		AA	20.1	19.1	22.6	22.6	16.2	17.9	**24.8**	14.2	15.7	**19.6**
ResNet56	SAT	Clean	56.3	55.1	**57.9**	47.1	43.9	**46.8**	46.4	30.9	32.2	**38.9**
		FGSM	29.4	30.6	28.4	**32.7**	25.3	27.4	**32.4**	20.6	22.4	**38.6**
		PGD^{20}	24.9	27.3	27.6	**31.2**	24.4	24.8	**30.9**	17.5	19.0	**36.4**
		PGD^{100}	24.9	27.1	27.9	**31.2**	24.5	25.0	**30.9**	17.5	19.1	**36.4**
		AA	22.1	23.1	24.0	**28.7**	18.9	20.4	**24.2**	13.8	16.7	**21.9**
	TRADES	Clean	54.1	52.8	53.0	**56.0**	48.6	49.3	**56.3**	40.5	39.6	**51.5**
		FGSM	32.2	27.6	30.4	**31.6**	26.8	28.1	**32.6**	21.4	21.6	**37.9**
		PGD^{20}	27.1	23.5	26.9	**27.6**	24.4	26.2	**30.1**	18.9	18.9	**35.4**
		PGD^{100}	27.1	23.4	26.5	**27.6**	23.9	26.0	**30.1**	18.5	18.8	**35.4**
		AA	23.9	20.1	21.6	**24.2**	20.1	23.6	**27.3**	15.4	16.7	**32.1**
	MART	Clean	48.2	47.7	48.9	**49.6**	45.9	**47.1**	41.8	38.6	38.4	**45.6**
		FGSM	33.1	30.4	33.4	33.4	28.5	30.6	**35.1**	21.9	24.8	**26.8**
		PGD^{20}	30.1	24.3	25.9	**26.6**	25.6	24.8	**31.4**	20.7	23.2	**25.4**
		PGD^{100}	31.4	24.1	25.7	**26.6**	25.2	24.6	**31.4**	20.6	23.0	**25.5**
		AA	27.1	23.6	**24.8**	23.1	**23.7**	23.1	18.6	16.4	**18.2**	16.8

(20%, 50%, and 90%), while the clean accuracy of GAL and CHIP can be properly maintained, their robust accuracy drops significantly. This phenomenon occurs because GAL and CHIP, as filter pruning methods, do not consider the robust characteristics of the model. Consequently, they can only ensure clean accuracy and fail to maintain robust accuracy against strong attacks, posing a significant security risk to the model.

Comparison with lightweight networks and search networks: We evaluate the performance of our method against lightweight networks and search networks in terms of accuracy and efficiency. The result is shown as Table 6. In terms of accuracy, FSRP achieves comparable or even superior results compared to lightweight networks. Despite being more compact, our method maintains a competitive level of accuracy, making it a promising option for resource-constrained scenarios. Regarding efficiency, our method offers advantages over search networks. Search networks require extensive computational resources and time-consuming search processes to find optimal architectures. Our method focuses on pruning and refining the existing network, leading to faster convergence and reduced computational complexity. Overall, our method strikes a balance between accuracy and efficiency, making it a favorable choice for various applications where both performance and computational resources are crucial factors to consider.

5.3. Results on ImageNet

Further, we demonstrate the effectiveness of our method using the more complex datasets ImageNet30 and ImageNet. We compare our method with Adv-LWM and HYDRA on VGG16/ResNet56 using the ImageNet30 and present the performance of the pruned models in Table 3. In most cases, FSRP is generally stronger than Adv-LWM and HYDRA. Even under the attack of ImageNet30 with a large amount of data, the pruned model with FSRP still retains strong defense capabilities.

Next, we present the pruning performance of our method on the ResNet50 model using the ImageNet dataset in Table 4. At pruning ratios of 20% and 50%, all three methods can maintain performance at an adequate level. However, at a high pruning ratio of 90%, the performance of all three pruning methods significantly decreases. This is likely due to the removal of too many parameters from the model, and the model's defense against adversarial attacks is further weakened by a large amount of data. However, FSRP still achieves higher clean and robust accuracy compared to Adv-LWM and HYDRA. This demonstrates that our method remains effective on more complex datasets.

5.4. Black-box attacks

White-box adversarial attacks are powerful methods used for robustness evaluation. However, they are often unrealistic when the model parameters are unknown to the attacker. Therefore, we also consider the model's robustness under black-box attacks, where the information about the target model is unknown to the attacker.

We use an unpruned robust model as the source model and a pruned model with a pruning ratio of 90% as the target model. We calculate the perturbation direction based on the gradients of the source model on input images and generate adversarial examples from the source model to evaluate the target model. We follow common settings from the literature [24] and apply transfer-based black-box attacks. The results are summarized in Tables 7 and 8. Under black-box attacks, the pruned models using FSRP demonstrate stronger defense capabilities compared to Adv-LWM and HYDRA. This also demonstrates the perfect transferability of the pruned models based on the FSRP pruning strategy.

5.5. Ablation study

To analyze the different factors affecting FSRP, we designed ablation studies in terms of both training targets and pruning strategies. The experimental results are reported in Table 9 and Fig. 6.

Table 3
Comparing FSRP with Adv-LWM and HYDRA robust pruning results on VGG16 and ResNet56 learned on ImageNet30.

Model	Adv. training	Attack type	Pre-trained	20%			50%			90%		
				Adv-LWM	HYDRA	FSRP	Adv-LWM	HYDRA	FSRP	Adv-LWM	HYDRA	FSRP
VGG16	TRADES	Clean	57.8	43.1	45.7	**48.2**	36.9	37.2	**42.3**	26.4	27.8	**36.7**
		FGSM	35.6	32.6	**36.1**	34.7	24.3	26.2	**28.5**	11.5	14.3	**22.4**
		PGD^{20}	33.3	30.6	**32.4**	31.6	19.4	22.4	**24.8**	10.5	12.8	**21.0**
		PGD^{100}	33.5	30.2	**31.8**	31.3	18.7	23.0	**24.3**	10.1	12.3	**20.3**
		AA	24.4	23.6	22.7	**24.9**	18.4	17.3	**19.6**	6.5	11.9	**16.3**
ResNet56	TRADES	Clean	65.2	42.9	43.6	**57.3**	32.5	37.1	**41.3**	24.6	24.3	**32.3**
		FGSM	36.4	26.4	29.4	**34.9**	18.6	19.7	**21.4**	8.9	10.3	**17.6**
		PGD^{20}	32.2	23.1	22.6	**29.7**	16.0	17.2	**18.3**	7.0	9.2	**15.3**
		PGD^{100}	34.4	23.4	22.1	**29.5**	16.3	17.4	**18.3**	7.0	9.2	**15.1**
		AA	26.1	15.4	18.3	**25.4**	**13.6**	12.7	11.5	4.5	7.3	**9.6**

Table 4
Comparing FSRP with Adv-LWM and HYDRA robust pruning results on ResNet50 learned on ImageNet.

Model	Adv. training	Attack type	Pre-trained	20%			50%			90%		
				Adv-LWM	HYDRA	FSRP	Adv-LWM	HYDRA	FSRP	Adv-LWM	HYDRA	FSRP
ResNet50	TRADES	Clean	61.6	54.3	56.4	**63.4**	50.7	53.9	**58.3**	23.6	27.2	**34.6**
		FGSM	37.6	35.1	36.9	**39.4**	32.8	32.9	**34.1**	11.9	14.3	**18.7**
		PGD^{20}	32.8	32.5	33.1	**38.0**	24.1	27.9	**28.7**	9.7	12.3	**14.7**
		PGD^{100}	32.4	31.8	33.0	**38.3**	24.0	27.5	**28.4**	9.3	12.3	**13.5**
		AA	28.8	30.2	31.6	**33.9**	20.2	24.5	**26.9**	4.8	10.1	**11.2**

Table 5
Results of robust pruning under different pruning criteria on CIFAR10. GAL and CHIP are pruning methods without considering robustness.

Model	Adv. training	Attack type	Pre-trained	20%			50%			90%		
				GAL	CHIP	FSRP	GAL	CHIP	FSRP	GAL	CHIP	FSRP
VGG16	TRADES	Clean	83.0	81.7	82.3	**84.0**	81.3	79.6	**83.0**	76.9	73.4	**81.1**
		FGSM	54.1	43.2	46.2	**65.5**	43.8	41.0	**60.6**	36.7	38.1	**56.7**
		PGD^{20}	54.0	36.5	39.1	**58.9**	34.6	35.9	**57.1**	32.4	31.0	**49.6**
		PGD^{100}	54.0	35.2	37.9	**58.9**	32.6	33.4	**57.0**	32.1	30.4	**49.7**
		AA	44.9	30.6	31.7	**58.9**	28.2	27.4	**52.7**	27.4	28.7	**49.8**

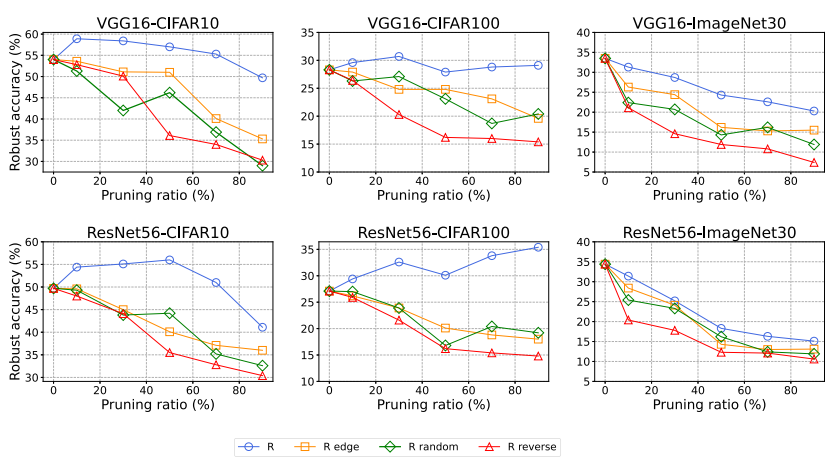

Fig. 6. Robust pruning results of VGG16/ResNet56 on CIAFR10/CIFAR100 / ImageNet30 under different pruning criteria.

Training target: We compare the clean and robust accuracy of ResNet56 with a pruning ratio of 90% on CIFAR10 under four different pre-training and fine-tuning objectives. Non-robust pre-training in this paper refers to pre-training the model using standard, and non-robust fine-tuning in this paper refers to fine-tuning the model using standard training. As shown in Table 9, using different pre-training and fine-tuning targets in the same model results in different clean and robust accuracies. The first row is the baseline models for standard training and fine-tuning with clean examples. The fourth row shows the training method taken in this study. The second row compared to the fourth row shows that the model without robust pre-training has a certain degree of degradation in robust accuracy. The third row compared to the fourth row shows that the robustness accuracy decreases significantly if the model is not fine-tuned for robustness. This indicates that pre-training and fine-tuning are effective in improving the robustness of the network.

Pruning strategy: We make some changes in the FSRP pruning strategy and propose three new variants to prove the effectiveness of the FSRP in this paper. We prune the VGG16/ResNet56 and evaluate the robust accuracy with PGD^{100}. The $\bar{R}$ positive order is the pruning criterion of FSRP; the edge pruning is pruning the filters at the two ends after sorting; the random pruning is the pruning of filters with random serial numbers; and the $\bar{R}$ reverse order is just the opposite of the pruning criterion of FSRP. The experimental results are shown in Fig. 6. We observe that different pruning criteria will lead to large differences in the robust accuracy of pruning models. (1) Compared

Table 6
Comparison of the number of network parameters and robustness accuracy for different structures on CIFAR10. The first 10 rows are the number of parameters and robust accuracy of the search network, the middle 5 rows are the number of parameters and robust accuracy of the lightweight network, and the last 3 rows are the results of robust pruning of VGG16, and ResNet56 by FSRP.

Model	Clean	FGSM	PGD^{20}	AA	Params
PNAS [56]	85.0	58.8	47.7	40.1	4.5M
NasNet [57]	80.6	54.2	50.2	39.2	4.3M
AmoebaNet [58]	83.4	54.4	42.9	35.3	3.2M
DARTS [59]	83.7	55.7	44.9	39.9	3.3M
P-DARTS [60]	82.6	53.2	42.7	37.2	3.4M
PC-DARTS [61]	83.9	52.6	41.9	39.2	3.6M
RACL [62]	83.6	57.2	50.0	44.1	3.6M
RobNet-free [63]	83.9	58.4	51.6	46.0	5.6M
DSRNA [64]	80.9	59.4	51.3	38.9	3.5M
AdvRush [65]	84.5	60.2	52.3	49.2	4.2M
SqueezeNet [42]	79.1	51.4	46.3	40.2	0.7M
ShuffleNetV2 [44]	79.1	51.9	47.1	41.6	1.2M
MobileNetV1 [45]	75.0	49.1	44.8	38.9	3.2M
MobileNetV2 [46]	81.6	54.6	50.2	43.7	2.2M
EfficientNet [47]	76.9	49.2	44.3	42.0	3.5M
VGG16 (FSRP)	81.6	56.4	51.8	49.7	1.4M
ResNet56 (FSRP)	83.2	60.8	56.7	50.4	0.5M

Table 7
Results of the model on CIFAR10 under black-box attacks.

Model	Method	FGSM	PGD^{100}
VGG16	Adv-LWM [21]	32.4	26.9
	HYDRA [22]	35.1	29.0
	FSRP	**41.8**	**36.3**
ResNet56	Adv-LWM [21]	26.5	23.9
	HYDRA [22]	27.0	22.5
	FSRP	**45.5**	**41.9**

Table 8
Results of the model on CIFAR100 under black-box attacks.

Model	Method	FGSM	PGD^{100}
VGG16	Adv-LWM [21]	36.1	35.4
	HYDRA [22]	33.0	32.7
	FSRP	**39.9**	**41.6**
ResNet56	Adv-LWM [21]	25.6	24.6
	HYDRA [22]	28.9	25.7
	FSRP	**42.1**	**40.9**

Table 9
Robust pruning results of ResNet56 on CIFAR10 dataset with different pre-training and fine-tuning.

Pre-train	Fine-Tune	Clean	FGSM	PGD^{20}	PGD^{100}	AA
✗	✗	**89.8**	40.9	10.0	9.7	9.2
✗	✓	75.2	50.3	46.1	45.4	42.6
✓	✗	88.8	41.1	10.7	10.3	9.2
✓	✓	78.2	**54.0**	**48.4**	**48.2**	**45.1**

with other pruning criteria, the robust accuracy of the model after FSRP pruning under the three pruning ratios (20%, 50%, and 90%) is the best, which shows that our pruning criterion is valid. (2) Under the $\bar{R}$ reverse pruning, the robust accuracy is the lowest, and more robust filters are subtracted, so more filters with weak robustness will focus on the high-frequency regions with strong vulnerability in the feature map. This will produce stronger perturbations to the deep neural network, thereby reducing robust accuracy. (3) The robust accuracy of the model decreases gradually as the pruning ratio increases after the edge pruning. The results of random pruning are associated with uncertainty.

6. Discussion

FSRP specifically targets data with explicit spatial structure, such as images. CNNs are commonly used for processing image data, where frequency characteristics, particularly low and high-frequency information, closely relate to the structure and details of the image. In contrast, the robustness of RNNs in handling sequential data, such as text, depends more on vocabulary choice, or grammatical structure, which may not be fully captured through frequency analysis.

In addition, CNN consists of convolutional layers and pooling layers, which are made up of weight-shared convolutional kernels (or filters) and non-linear activation functions (such as ReLU). In contrast, Transformers [66] consist of self-attention layers (in the encoder or decoder) and feed-forward fully connected layers. Especially like ViT [67], a vision model based on Transformer, it can perform image classification without using convolutional neural networks. Since our work is primarily focused on CNN research, it seems difficult to apply it to ViT.

7. Conclusion

In this paper, we reveal the robustness characteristics of the deep feature space of the model. The robust model focuses more on the low-frequency regions of the feature map of the adversarial examples, which indicates the low-frequency regions in the feature map represent robustness, while the high-frequency regions reflect weaker robustness. These findings help us to construct an effective non-robust indicator, *i.e.*, the average ratio of the low-frequency component to the high-frequency component of the feature map. We further use this indicator as a filter pruning criterion to guide our proposed robust pruning algorithm. The experiments prove that our robust pruning algorithm can more effectively improve the robustness than other baselines. In the future, we will extend this approach to more neural network architectures and tasks.

CRediT authorship contribution statement

Yaguan Qian: Investigation. **Wenzhuo Huang:** Methodology, Investigation. **Qinqin Yu:** Investigation. **Tengteng Yao:** Investigation. **Xiang Ling:** Investigation. **Bin Wang:** Supervision, Funding acquisition. **Zhaoquan Gu:** Supervision. **Yanchun Zhang:** Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Bin Wang reports financial support was provided by the National Natural Science Foundation of China. Yaguan Qian reports financial support was provided by Zhejiang Provincial Natural Science Foundation of China.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (62476250, 62472335, 62372137), in part by the Key Program of Zhejiang Provincial Natural Science Foundation of China (LZ22F020007), and in part by the Foundation of Zhejiang Key Laboratory of Multi-dimensional Perception Technology Application and Cybersecurity (HIK2022008).

References

[1] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.

[2] C. Szegedy, S. Ioffe, V. Vanhoucke, A.A. Alemi, Inception-v4, inception-resnet and the impact of residual connections on learning, in: Thirty-First AAAI Conference on Artificial Intelligence, 2017.

[3] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie, Feature pyramid networks for object detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 2117–2125.

[4] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: Unified, real-time object detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 779–788.

[5] G. Litjens, T. Kooi, B.E. Bejnordi, A.A.A. Setio, F. Ciompi, M. Ghafoorian, J.A. Van Der Laak, B. Van Ginneken, C.I. Sánchez, A survey on deep learning in medical image analysis, Med. Image Anal. 42 (2017) 60–88.

[6] X. Shi, M. Sapkota, F. Xing, F. Liu, L. Cui, L. Yang, Pairwise based deep ranking hashing for histopathology image classification and retrieval, Pattern Recognit. 81 (2018) 14–22.

[7] M.G.S. Murshed, C. Murphy, D. Hou, N. Khan, G. Ananthanarayanan, F. Hussain, Machine learning at the network edge: A survey, ACM Comput. Surv. 54 (8) (2022) 170:1–170:37.

[8] W. Yu, F. Liang, X. He, W.G. Hatcher, C. Lu, J. Lin, X. Yang, A survey on the edge computing for the Internet of Things, IEEE Access 6 (2017) 6900–6919.

[9] Y. Qian, Y. Guo, Q. Shao, J. Wang, B. Wang, Z. Gu, X. Ling, C. Wu, EI-MTD: Moving target defense for edge intelligence against adversarial attacks, ACM Trans. Priv. Secur. 25 (3) (2022) 1–24.

[10] Z. Lu, S. Rallapalli, K. Chan, T. La Porta, Modeling the resource requirements of convolutional neural networks on mobile devices, in: Proceedings of the 25th ACM International Conference on Multimedia, 2017, pp. 1663–1671.

[11] I.J. Goodfellow, J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, in: 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings, 2015.

[12] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I.J. Goodfellow, R. Fergus, Intriguing properties of neural networks, in: 2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14–16, 2014, Conference Track Proceedings, 2014.

[13] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, Comput. Sci. 14 (7) (2015) 38–39.

[14] M. Goldblum, L. Fowl, S. Feizi, T. Goldstein, Adversarially robust distillation, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, (04) 2020, pp. 3996–4003.

[15] B. Huang, M. Chen, Y. Wang, J. Lu, M. Cheng, W. Wang, Boosting accuracy and robustness of student models via adaptive adversarial distillation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 24668–24677.

[16] S. Zhao, J. Yu, Z. Sun, B. Zhang, X. Wei, Enhanced accuracy and robustness via multi-teacher adversarial distillation, in: European Conference on Computer Vision, Springer, 2022, pp. 585–602.

[17] F. Ayaz, I. Zakariyya, J. Cano, S.L. Keoh, J. Singer, D. Pau, M. Kharbouche-Harrari, Improving robustness against adversarial attacks with deeply quantized neural networks, in: 2023 International Joint Conference on Neural Networks, IJCNN, IEEE, 2023, pp. 1–8.

[18] J.-H. Luo, J. Wu, W. Lin, Thinet: A filter level pruning method for deep neural network compression, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 5058–5066.

[19] Y. Sui, M. Yin, Y. Xie, H. Phan, S. Aliari Zonouz, B. Yuan, CHIP: Channel independence-based pruning for compact neural networks, Adv. Neural Inf. Process. Syst. 34 (2021) 24604–24616.

[20] S. Lin, R. Ji, C. Yan, B. Zhang, L. Cao, Q. Ye, F. Huang, D. Doermann, Towards optimal structured cnn pruning via generative adversarial learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 2790–2799.

[21] V. Sehwag, S. Wang, P. Mittal, S. Jana, Towards compact and robust deep neural networks, 2019, arXiv preprint arXiv:1906.06110.

[22] V. Sehwag, S. Wang, P. Mittal, S. Jana, Hydra: Pruning adversarially robust neural networks, Adv. Neural Inf. Process. Syst. 33 (2020) 19655–19666.

[23] D. Yin, R.G. Lopes, J. Shlens, E.D. Cubuk, J. Gilmer, A Fourier perspective on model robustness in computer vision, in: Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada, 2019, pp. 13255–13265.

[24] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, in: 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30–May 3, 2018, Conference Track Proceedings, 2018.

[25] F. Croce, M. Hein, Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks, in: International Conference on Machine Learning, PMLR, 2020, pp. 2206–2216.

[26] F. Croce, M. Hein, Minimally distorted adversarial examples with a fast adaptive boundary attack, in: International Conference on Machine Learning, PMLR, 2020, pp. 2196–2205.

[27] M. Andriushchenko, F. Croce, N. Flammarion, M. Hein, Square attack: A query-efficient black-box adversarial attack via random search, in: European Conference on Computer Vision, Springer, 2020, pp. 484–501.

[28] Y. Liu, X. Chen, C. Liu, D. Song, Delving into transferable adversarial examples and black-box attacks, in: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings, 2017.

[29] Y. Dong, F. Liao, T. Pang, H. Su, J. Zhu, X. Hu, J. Li, Boosting adversarial attacks with momentum, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 9185–9193.

[30] C. Xie, Z. Zhang, Y. Zhou, S. Bai, J. Wang, Z. Ren, A.L. Yuille, Improving transferability of adversarial examples with input diversity, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 2730–2739.

[31] X. Liu, Y. Li, C. Wu, C. Hsieh, Adv-BNN: Improved adversarial defense through robust Bayesian neural network, in: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, la, USA, May 6–9, 2019, 2019.

[32] Z. Gong, W. Wang, Adversarial and clean data are not twins, in: Proceedings of the Sixth International Workshop on Exploiting Artificial Intelligence Techniques for Data Management, AiDM@SIGMOD 2023, Seattle, WA, USA, 18 June 2023, 2023, pp. 6:1–6:5.

[33] L. Pan, C.-W. Hang, A. Sil, S. Potdar, Improved text classification via contrastive adversarial training, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, (10) 2022, pp. 11130–11138.

[34] H. Zhang, Y. Yu, J. Jiao, E. Xing, L. El Ghaoui, M. Jordan, Theoretically principled trade-off between robustness and accuracy, in: International Conference on Machine Learning, PMLR, 2019, pp. 7472–7482.

[35] Y. Wang, D. Zou, J. Yi, J. Bailey, X. Ma, Q. Gu, Improving adversarial robustness requires revisiting misclassified examples, in: 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26–30, 2020.

[36] H. Huang, Y. Wang, S. Erfani, Q. Gu, J. Bailey, X. Ma, Exploring architectural ingredients of adversarially robust deep neural networks, Adv. Neural Inf. Process. Syst. 34 (2021) 5545–5559.

[37] S. Ye, K. Xu, S. Liu, H. Cheng, J.-H. Lambrechts, H. Zhang, A. Zhou, K. Ma, Y. Wang, X. Lin, Adversarial robustness vs. model compression, or both? in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 111–120.

[38] S. Gui, H. Wang, H. Yang, C. Yu, Z. Wang, J. Liu, Model compression with adversarial robustness: A unified optimization framework, in: Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada, 2019, pp. 1283–1294.

[39] A. Jordao, H. Pedrini, On the effect of pruning on adversarial robustness, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 1–11.

[40] Y. Guo, C. Zhang, C. Zhang, Y. Chen, Sparse DNNs with improved adversarial robustness, in: Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3–8, 2018, MontrÉAl, Canada, 2018, pp. 240–249.

[41] H. Xu, C. Caramanis, S. Mannor, Sparse algorithms are not stable: A no-free-lunch theorem, IEEE Trans. Pattern Anal. Mach. Intell. 34 (1) (2011) 187–193.

[42] F.N. Iandola, S. Han, M.W. Moskewicz, K. Ashraf, W.J. Dally, K. Keutzer, SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and< 0.5 MB model size, 2016, arXiv preprint arXiv:1602.07360.

[43] M.J. Narasimha, A.M. Peterson, On the computation of the discrete cosine transform, IEEE Trans. Commun. 26 (6) (1978) 934–936.

[44] N. Ma, X. Zhang, H.-T. Zheng, J. Sun, Shufflenet v2: Practical guidelines for efficient cnn architecture design, in: Proceedings of the European Conference on Computer Vision, ECCV, 2018, pp. 116–131.

[45] G. Andrew, Z. Menglong, et al., Efficient convolutional neural networks for mobile vision applications, Mobilenets 10 (2017) 151.

[46] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.C. Chen, MobileNetV2: Inverted residuals and linear bottlenecks, in: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2018.

[47] M. Tan, Q. Le, Efficientnet: Rethinking model scaling for convolutional neural networks, in: International Conference on Machine Learning, PMLR, 2019, pp. 6105–6114.

[48] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, A. Madry, Adversarial examples are not bugs, they are features, in: Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada, 2019, pp. 125–136.

[49] A. Li, Y. Wang, Y. Guo, Y. Wang, Adversarial examples are not real features, in: Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, la, USA, December 10–16, 2023.

[50] H. Yang, J. Zhang, H. Dong, N. Inkawhich, A. Gardner, A. Touchet, W. Wilkes, H. Berry, H. Li, DVERGE: Diversifying vulnerabilities for enhanced robust generation of ensembles, Adv. Neural Inf. Process. Syst. 33 (2020) 5505–5515.

[51] P. Wang, W. Zheng, T. Chen, Z. Wang, Anti-oversmoothing in deep vision transformers via the Fourier domain analysis: From theory to practice, in: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022.

[52] H. Tanaka, D. Kunin, D.L. Yamins, S. Ganguli, Pruning neural networks without any data by iteratively conserving synaptic flow, Adv. Neural Inf. Process. Syst. 33 (2020) 6377–6389.

[53] A. Krizhevsky, Learning Multiple Layers of Features from Tiny Images, (Master's thesis), University of Tront, 2009.

[54] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, Imagenet: A large-scale hierarchical image database, in: 2009 IEEE Conference on Computer Vision and Pattern Recognition, Ieee, 2009, pp. 248–255.

[55] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, in: Y. Bengio, Y. LeCun (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings, 2015.

[56] C. Liu, B. Zoph, M. Neumann, J. Shlens, W. Hua, L.-J. Li, L. Fei-Fei, A. Yuille, J. Huang, K. Murphy, Progressive neural architecture search, in: Proceedings of the European Conference on Computer Vision, ECCV, 2018, pp. 19–34.

[57] B. Zoph, V. Vasudevan, J. Shlens, Q.V. Le, Learning transferable architectures for scalable image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 8697–8710.

[58] E. Real, A. Aggarwal, Y. Huang, Q.V. Le, Regularized evolution for image classifier architecture search, in: Proceedings of the Aaai Conference on Artificial Intelligence, vol. 33, (01) 2019, pp. 4780–4789.

[59] H. Liu, K. Simonyan, Y. Yang, DARTS: differentiable architecture search, in: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, la, USA, May 6–9, 2019.

[60] X. Chen, L. Xie, J. Wu, Q. Tian, Progressive differentiable architecture search: Bridging the depth gap between search and evaluation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 1294–1303.

[61] Y. Xu, L. Xie, X. Zhang, X. Chen, G. Qi, Q. Tian, H. Xiong, PC-DARTS: partial channel connections for memory-efficient architecture search, in: 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26–30, 2020.

[62] M. Dong, Y. Li, Y. Wang, C. Xu, Adversarially robust neural architectures, 2020, arXiv preprint arXiv:2009.00902.

[63] M. Guo, Y. Yang, R. Xu, Z. Liu, D. Lin, When NAS meets robustness: In search of robust architectures against adversarial attacks, in: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13–19, 2020, pp. 628–637.

[64] R. Hosseini, X. Yang, P. Xie, Dsrna: Differentiable search of robust neural architectures, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 6196–6205.

[65] J. Mok, B. Na, H. Choe, S. Yoon, AdvRush: Searching for adversarially robust neural architectures, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 12322–12332.

[66] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).

[67] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, An image is worth 16x16 words: Transformers for image recognition at scale, in: 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3–7, 2021.

Yaguan Qian received a Ph.D. in Computer Science from Zhejiang University in 2014. He is currently a full professor at the School of Science, Zhejiang University of Science and Technology. He has authored or co-authored more than 40 papers in peer-reviewed international conferences and journals, like AAAI, ECCV, ICCV, TIFS, TNNLS, and TOPS. His main research interests include AI security, multimodal large models, machine learning, and machine vision.

Wenzhuo Huang received the B.S. degree in computer science from Sanjiang University in 2021. She is currently pursuing an M.S. degree at the School of Science, Zhejiang University of Science and Technology. Her research focuses on adversarial learning and computer vision.

Qinqin Yu received a Bachelor of Science from Hunan University of Technology and Business in 2021. She is currently pursuing an M.S. degree at the School of Science, Zhejiang University of Science and Technology. Her research focuses on adversarial learning and computer vision.

Tengteng Yao received Ph.D. in computational mathematics from Xiamen University in 2016. She is currently an associate professor at the School of Science, Zhejiang University of Science and Technology. She has authored or co-authored more than several papers in peer-reviewed international journals and conferences, like SIAM Journal on Matrix Analysis and Applications, Numerical Linear Algebra with Applications, BIT Numerical Mathematics and Numerical Algorithms. She is currently a reviewer for the Mathematical Reviews.

Xiang Ling is currently a research faculty at the Institute of Software, Chinese Academy of Sciences. He received his Ph.D. degree from Zhejiang University. His research focuses on data-driven security, AI security and program analysis. His work has been published in top-ranked conferences and journals, like IEEE S&P, INFOCOM, ICDCS, Black Hat Asia, TNNLS, and TKDD.

Bin Wang is a professor with the Zhejiang Key Laboratory of Multi-dimensional Perception Technology, Application and Cybersecurity and Zhejiang University. He received his Ph.D. degree from the China National Digital Switching System Engineering & Technological R&D Center in 2010. His research interests mainly include Internet of Things security, cryptography, artificial intelligence security, new network security architecture, and so on.

Zhaoquan Gu received his bachelor's degree in Computer Science from Tsinghua University (2011) and a Ph.D. degree in Computer Science from Tsinghua University (2015). He is currently a professor at the School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), China. He is also a professor in the Department of New Networks, Peng Cheng Laboratory, Shenzhen, China. His research interests include cyberspace security, cyber range, big data analysis, and artificial intelligence security.

Yanchun Zhang received a Ph.D. degree in computer science, the University of Queensland, in 1991. He is currently the Director of the IT Program (Data Science and AI) with the Institute for Sustainable Industries and Liveable Cities, VU Research, Victoria University, Melbourne, VIC, Australia, and coordinates a multidisciplinary e-research program across Victoria University. He is also an International Research Leader in databases, data mining, health informatics, Web information systems, and Web services. He has published over 300 research articles in international journals and conference proceedings, including ACM Transactions on Computer and Human Interaction (TOCHI), the IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING (TKDE), VLDBJ, SIGMOD, and ICDE conferences, and a dozen of books and journal special issues in the related areas. Dr. Zhang is a Founding Editor and the Editor-in-Chief of the World Wide Web Journal (Springer) and Health Information Science and Systems Journal (Springer). He is also the Chairman of the International Web Information Systems Engineering Society.